SQL vs. NoSQL

1. Data Model

- **SQL**
- Uses **tables** (rows & columns)
- Fixed **schema**
- **Example (Table):**

```
Students
---------------------------------
ID | Name   | Age | Department
1  | Ahmed  | 20  | CS
2  | Sara   | 21  | IT
```

- **NoSQL**
- Uses **flexible data models** (document, key-value, column, graph)
- **Schema-less**
- **Example (Document – MongoDB):**

```json
{
  "id": 1,
  "name": "Ahmed",
  "age": 20,
  "department": "CS",
  "courses": ["DB", "AI"]
}
```

Note: If in the next object you need to add more attributes such as birth-date, you can add it simply without any need to change in db structure “schema-less”.

2. **Schema**

- **SQL:** Schema must be defined **before** inserting data

```sql
CREATE TABLE Students (
  ID INT,
  Name VARCHAR(50),
  Age INT
);
```

- **NoSQL:** No predefined schema, Each document can be different.

```json
{ "name": "Sara", "email": "sara@gmail.com" }
{ "name": "Ali", "phone": "010..." }
```

3. Consistency & Transactions

- **SQL**

Strong **ACID** properties

Always consistent

- **Example:**
  Bank transfer (money must not be lost)
- **NoSQL**

Uses **BASE** (eventual consistency)

- **Example:**
  Social media likes may update slightly later

4. Relationships

- **SQL**

Supports **JOINs**

```sql
SELECT s.name, c.course
FROM Students s
JOIN Courses c ON s.id = c.student_id;
```

- **NoSQL**

No JOINs (data is often embedded)

```json
{
  "name": "Ahmed",
  "courses": ["DB", "AI"]
}
```

Feature	SQL Project Example	NoSQL Project Example
Data type	Structured	Semi-structured / unstructured
Scaling	Vertical	Horizontal
Consistency	ACID	Eventual (BASE)
Users	Thousands	Millions
Examples	Banking, ERP, E-commerce orders	Social media, Netflix analytics, IoT, Amazon catalog

Think????

Use Case	SQL	NoSQL
Banking systems		
E-commerce orders		
Social media		
Big data / real-time		
IoT data		

Use Case	SQL	NoSQL
Banking systems	✅	❌
E-commerce orders	✅	❌
Social media	❌	✅
Big data / real-time	❌	✅
IoT data	❌	✅

- **SQL Databases**

MySQL
PostgreSQL
Oracle
SQL Server

- **NoSQL Databases**

MongoDB
Cassandra
Redis
DynamoDB
Neo4j

Projects Using SQL Databases

- **Example 1: Banking Systems**
- **Why SQL:**
  - Strong **ACID compliance** → critical for transactions
  - Structured data → accounts, customers, transactions
- **Project Example:**
  - Online banking application
  - Funds transfer, account balance updates, transaction history
- **Example 2: E-Commerce Websites**
- **Why SQL:**
  - Structured orders, inventory, users
  - Relationships between tables (customers ↔ orders ↔ products)
- **Project Example:**
  - Querying order history, generating invoices

- **Summary for SQL Projects**
- Projects that need **transactional integrity, structured data, and complex relationships**

Projects Using NoSQL Databases

- **Example 1: Social Media Platforms**
- **Why NoSQL:**
  - Huge volume of user-generated data
  - Schema flexibility (posts, comments, likes, media files)
  - Horizontal scaling for millions of users
- **Project Example:**
  - Facebook posts/comments, Instagram feed
  - MongoDB storing posts with varying fields per user
- **Example 2: Real-Time Analytics / Big Data**
- **Why NoSQL:**
  - High write throughput
  - Distributed architecture → handles **terabytes of data**
- **Project Example:**
  - Netflix analytics (user watch history, recommendations)
  - Log streaming for clickstreams and metrics